TP4 – Breaking the Classics

Information Security

Ahmed Khalil El Atri

1 Warm-Up – Caesar Cipher

Code

```python
def caesar_cipher(text, k):
    result = ""
    for char in text:
        if char.isalpha():
            shift = k % 26
            if char.islower():
                result += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))
            else:
                result += chr((ord(char) - ord('A') + shift) % 26 + ord('A'))
        else:
            result += char
    return result
print(caesar_cipher("crypto is fun until the professor says quiz", 4))
def caesar_decipher(text, k):
    return caesar_cipher(text, -k)
print(caesar_decipher("GWCL JMBBMZ VWB JM CAQVO BWWTA BW AWTDM BPQA", 8))
for i in range(1, 26):
    print(f"Shift {i}: {caesar_decipher('ESP VPJ TD FYVYZHY ECJ MCFEP QZCNP', i)}")
```

Output

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\User\Desktop\CC\Info Sec\TP> & D:\Anaconda\python.exe "c:/Users/User/Desktop/CC/Info Sec/TP/TP-Crypto/warmup.py"
gvctxs mw jyr yrxmp xli tvsjiwwsv wecw uymd
YOUD BETTER NOT BE USING TOOLS TO SOLVE THIS
Shift 1: DRO UOI SC EXUXYGX DBI LBEDO PYBMO
Shift 2: CQN TNH RB DWTWXFW CAH KADCN OXALN
Shift 3: BPM SMG QA CVSVWEV BZG JZCBM NWZKM
Shift 4: AOL RLF PZ BURUVDU AYF IYBAL MVYJL
Shift 5: ZNK QKE OY ATQTUCT ZXE HXAZK LUXIK
Shift 6: YMJ PJD NX ZSPSTBS YWD GWZYJ KTWHJ
Shift 7: XLI OIC MW YRORSAR XVC FVYXI JSVGI
Shift 8: WKH NHB LV XQNQRZQ WUB EUXWH IRUFH
Shift 9: VJG MGA KU WPMPQYP VTA DTWVG HQTEG
Shift 10: UIF LFZ JT VOLOPXO USZ CSVUF GPSDF
Shift 11: THE KEY IS UNKNOWN TRY BRUTE FORCE
Shift 12: SGD JDX HR TMJMNVM SQX AQTSD ENQBD
Shift 13: RFC ICW GQ SLILMUL RPW ZPSRC DMPAC
Shift 14: QEB HBV FP RKHKLTK QOV YORQB CLOZB
Shift 15: PDA GAU EO QJGJKSJ PNU XNQPA BKNYA
Shift 16: OCZ FZT DN PIFIJRI OMT WMPOZ AJMXZ
Shift 17: NBY EYS CM OHEHIQH NLS VLONY ZILWY
Shift 18: MAX DXR BL NGDGHPG MKR UKNMX YHKVX
Shift 19: LZW CWQ AK MFCFGOF LJQ TJMLW XGJUW
Shift 20: KYV BVP ZJ LEBEFNE KIP SILKV WFITV
Shift 21: JXU AUO YI KDADEMD JHO RHKJU VEHSU
Shift 22: IWT ZTN XH JCZCDLC IGN QGJIT UDGRT
Shift 23: HVS YSM WG IBYBCKB HFM PFIHS TCFQS
Shift 24: GUR XRL VF HAXABJA GEL OEHGR SBEPR
Shift 25: FTQ WQK UE GZWZAIZ FDK NDGFQ RADOQ
PS C:\Users\User\Desktop\CC\Info Sec\TP>
```

Caesar encryption, decryption, and the 25 non-zero shifts. **Result.** The unknown key is $K = 11$, which reveals `THE KEY IS UNKNOWN TRY BRUTE FORCE`. Caesar has 26 possible shifts, including the identity shift; testing them is practical.

2 Room 1 – The Letter Wall

2.1 Encryption and Decryption

Code

```
#substition cipher
def substitution_cipher(text, cipher_alphabet):
    result = ""
    for char in text:
        if char.isalpha():
            if char.islower():
                index = ord(char) - ord('a')
                result += cipher_alphabet[index].lower()
            else:
                index = ord(char) - ord('A')
                result += cipher_alphabet[index].upper()
        else:
            result += char
    return result
def substitution_decipher(text, cipher_alphabet):
    result = ""
    for char in text:
        if char.isalpha():
            if char.islower():
                index = cipher_alphabet.lower().index(char)
                result += chr(index + ord('a'))
            else:
                index = cipher_alphabet.upper().index(char)
                result += chr(index + ord('A'))
        else:
            result += char
    return result
example_cipher_alphabet = "QMJZTGFKPWLSBOXNCRYEVHIADU"
print(substitution_cipher("KEEP THE SECRET SAFE", example_cipher_alphabet))
print(substitution_decipher("ERVYE DXVR SXFPJ", example_cipher_alphabet))
```

Results. Encryption gives `LTTN EKT YTJRTE YQGT`; decryption gives `TRUST YOUR LOGIC`. These results appear at the top of the first output screenshot below.

2.2 Frequency Analysis and Message Recovery

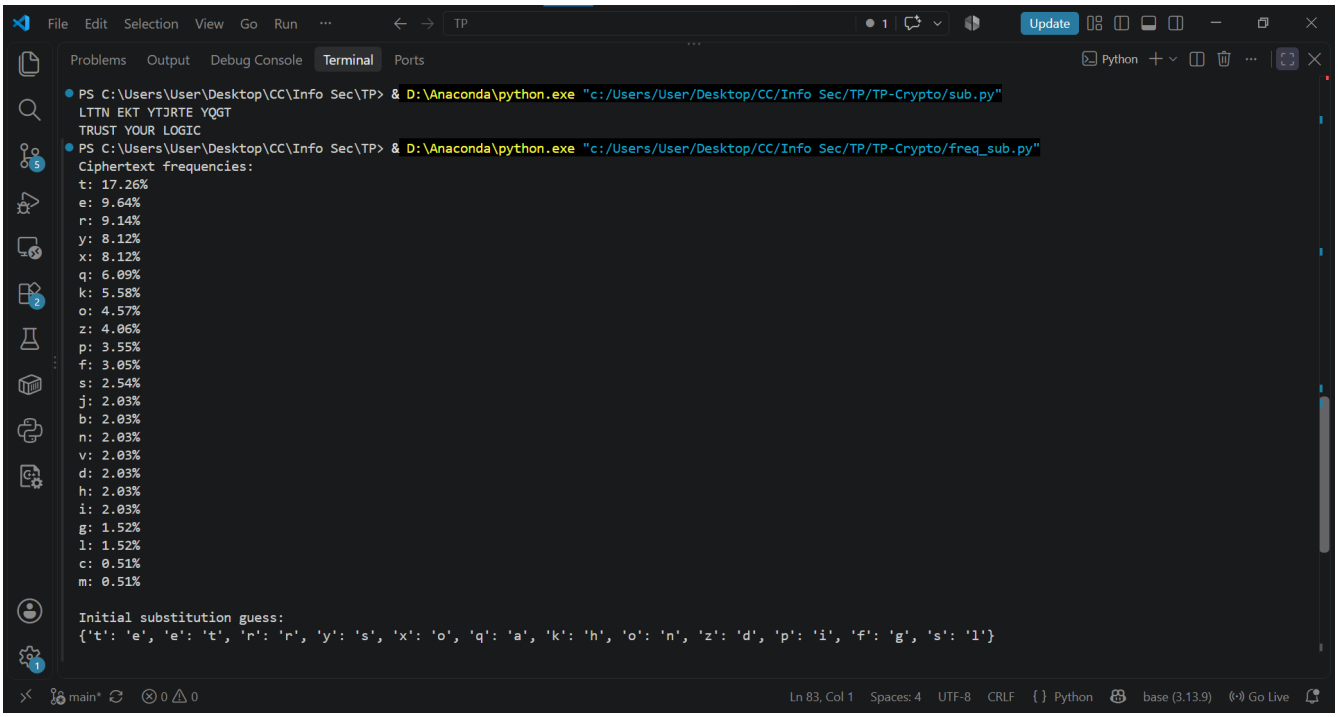
A substitution key has $26! \approx 4.03 \times 10^{26}$ possibilities. Instead of brute force, ciphertext frequencies suggest partial mappings, which are refined using repeated words and context.

Code

```
# Room 1 - Frequency substitution decipher
def frequency_substitution_decipher(text):
    original_text = text
    clean_text = ''.join(filter(str.isalpha, text)).lower()
    frequency = {}
    for char in clean_text:
        if char in frequency:
            frequency[char] += 1
        else:
            frequency[char] = 1
    total = sum(frequency.values())
    for char in frequency:
        frequency[char] = (frequency[char] / total) * 100
    sorted_frequency = sorted( frequency.items(), key=lambda x: x[1], reverse=True )
    print("Ciphertext frequencies:")
    for char, freq in sorted_frequency:
        print(f"{char}: {freq:.2f}%")
    # Hint given in the TP
    plaintext_frequency_order = ['e', 't', 'r', 's', 'o', 'a',
                                 'h', 'n', 'd', 'i', 'g', 'l']
    # first automatic frequency-based guess
    substitution = {}
    for i in range(min(len(sorted_frequency), len(plaintext_frequency_order))):
        cipher_char = sorted_frequency[i][0]
        plain_char = plaintext_frequency_order[i]
        substitution[cipher_char] = plain_char
    print("\nInitial substitution guess:")
    print(substitution)
```

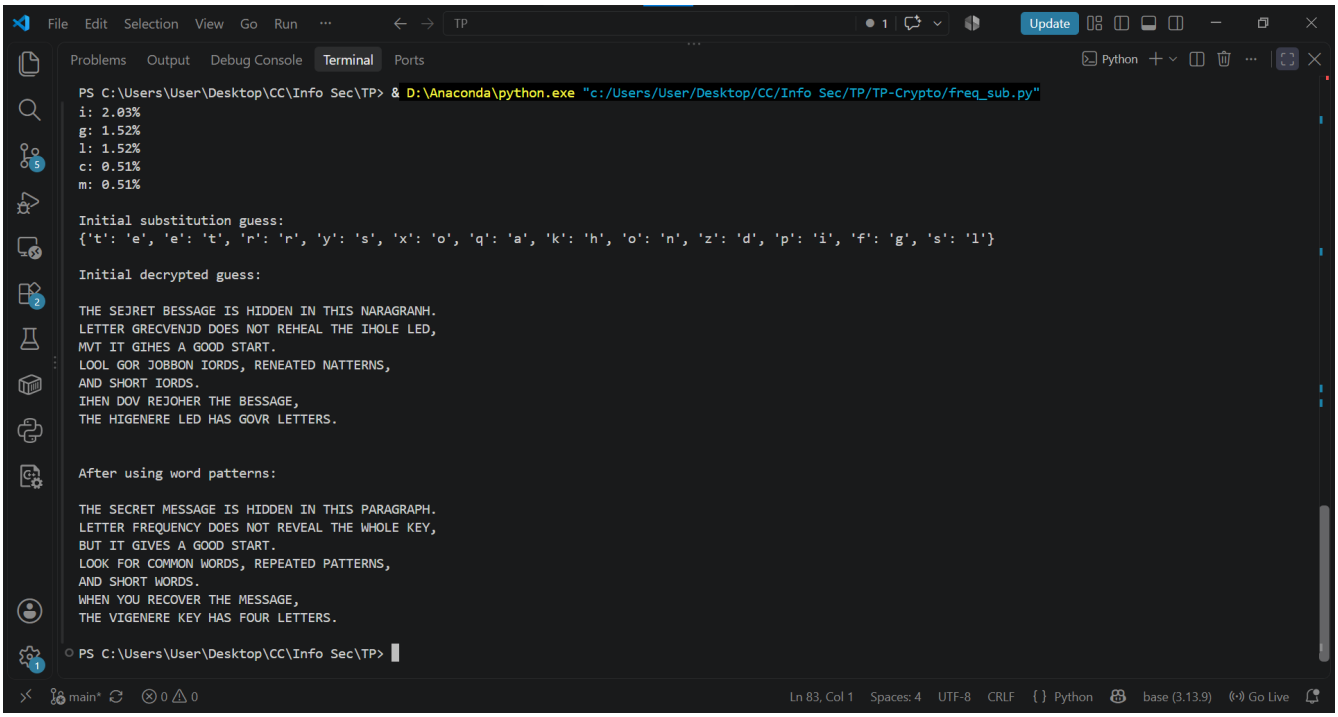
```
    # apply initial guess
    result = ""
    for char in original_text:
        if char.isalpha():
            lower_char = char.lower()
            if lower_char in substitution:
                replacement = substitution[lower_char]
                if char.isupper():
                    result += replacement.upper()
                else:
                    result += replacement
            else:
                result += char
        else:
            result += char
    print("\nInitial decrypted guess:")
    print(result)
    # Improve mapping using word patterns and context
    #EKT -> THE
    substitution['e'] = 't'
    substitution['k'] = 'h'
    substitution['t'] = 'e'
    # EKT YTJRTE BTYYQFT -> THE SECRET MESSAGE
    substitution['y'] = 's'
    substitution['j'] = 'c'
    substitution['r'] = 'r'
    substitution['b'] = 'm'
    substitution['q'] = 'a'
    substitution['f'] = 'g'
    # PY -> IS
    substitution['p'] = 'i'
    # KPZZTO -> HIDDEN
    substitution['z'] = 'd'
    substitution['o'] = 'n'
    # NQRQFRQNK -> PARAGRAPH
    substitution['n'] = 'p'
    # ZXTY -> DOES
    substitution['x'] = 'o'
    # FPHTY -> GIVES
    substitution['h'] = 'v'
    # MVE -> BUT
    substitution['m'] = 'b'
    substitution['v'] = 'u'
    #STEETR -> LETTER
    substitution['s'] = 'l'
    # GRTCVTOJD -> FREQUENCY
    substitution['c'] = 'q'
    substitution['d'] = 'y'
    #SXXL GXR JXBBXO IXRZY -> LOOK FOR COMMON WORDS
    substitution['l'] = 'k'
    substitution['g'] = 'f'
    substitution['i'] = 'w'
    # decrypt again using confirmed mappings
    result = ""
    for char in original_text:
        if char.isalpha():
            lower_char = char.lower()
            if lower_char in substitution:
                replacement = substitution[lower_char]
                if char.isupper():
                    result += replacement.upper()
                else:
                    result += replacement
            else:
                # leave unknown ciphertext letters unchanged
                result += char
        else:
            result += char
    print("\nAfter using word patterns:")
    print(result)
    return frequency, substitution
ciphertext = """
EKT YTJRTE BTYYQFT PY KPZZTO PO EKPY NQRQFRQNK.
STEETR GRTCVTOJD ZXTY OXE RTHTQS EKT IKXST LTD,
MVE PE FPHTY Q FXXZ YEQRE.
SXXL GXR JXBBXO IXRZY, RTNTQETZ NQEETROY,
QOZ YKXRE IXRZY.
IKTO DXV RTJXHTR EKT BTYYQFT,
EKT HPFTOTRT LTD KQY GXVR STEETRY.
"""
frequency_substitution_decipher(ciphertext)
```

Output – Frequencies and Initial Mapping



Substitution encryption/decryption, letter frequencies, and the first mapping from the provided hint. **Remark.** The hint $E > T > R > S/O > A > H > N > D > I > G > L$ supplies an initial guess. Word patterns such as `EKT`→`THE` and `GRTCVTOJD`→`FREQUENCY` complete the mapping for the observed letters.

Output – Recovered Message



Initial frequency-based guess followed by the message recovered through word patterns. **Clue for Room 2.** `THE VIGENERE KEY HAS FOUR LETTERS`.

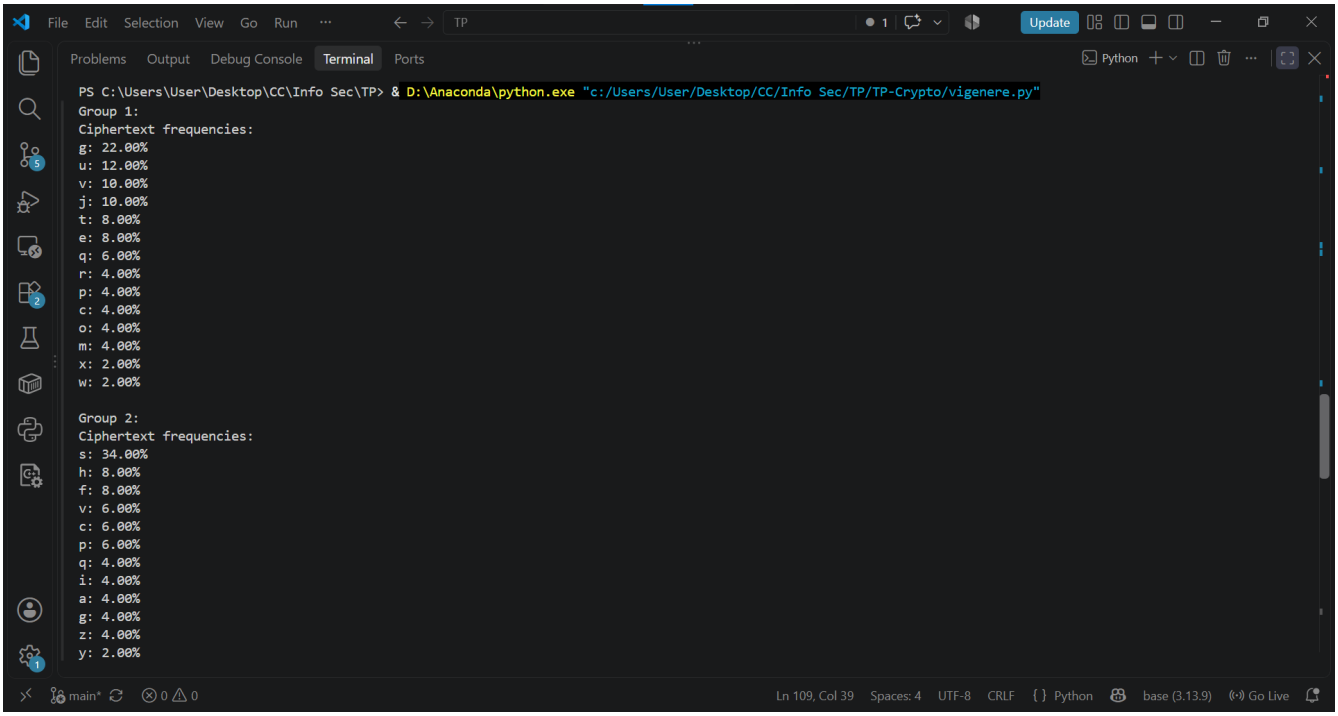
3 Room 2 – Break the Vigenère Cipher

Code

The ciphertext letters are split into four groups after removing spaces and line breaks. Each group uses one Caesar shift. Assuming its most frequent letter represents `E` gives the corresponding key letter.

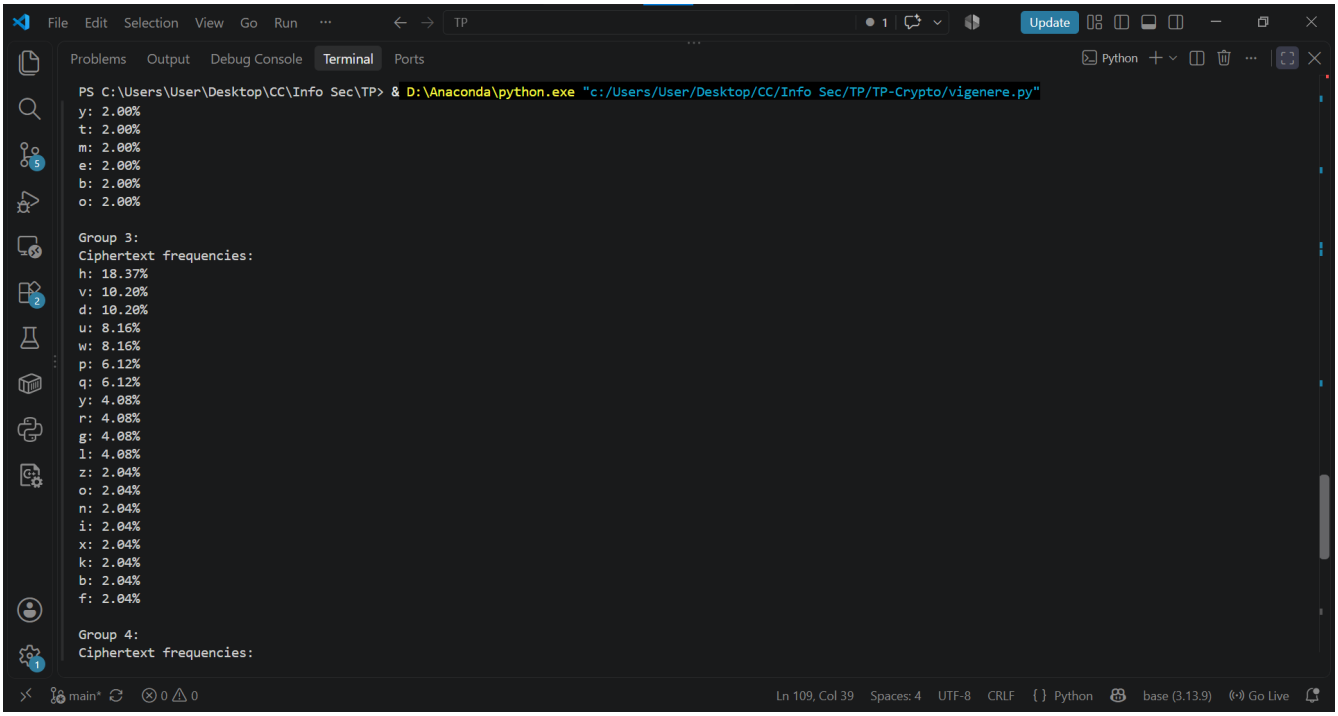
```
def frequency_analysis(ciphertext):
    frequency = {}
    total = 0
    for char in ciphertext:
        if char.isalpha():
            char = char.lower()
            frequency[char] = frequency.get(char, 0) + 1
            total += 1
    for char in frequency:
        frequency[char] = (frequency[char] / total) * 100
    sorted_frequency = sorted(
        frequency.items(),
        key=lambda x: x[1],
        reverse=True
    )
    print("Ciphertext frequencies:")
    for char, freq in sorted_frequency:
        print(f"{char}: {freq:.2f}%")
    return sorted_frequency
def vigenere_extract_key(ciphertext, key_length):
    # Remove spaces, punctuation and newlines first
    clean_ciphertext = ''.join(
        char.lower()
        for char in ciphertext
        if char.isalpha()
    )
    # Split into groups
    groups = ['' for _ in range(key_length)]
    for i, char in enumerate(clean_ciphertext):
        groups[i % key_length] += char
    # Frequency analysis
    for i, group in enumerate(groups):
        print(f"\nGroup {i + 1}:")
        frequency_analysis(group)
    # Assume most frequent plaintext letter is E
    key = ""
    for group in groups:
        most_frequent = max(group, key=group.count)
        # both are lowercase now
        shift = (ord(most_frequent) - ord('e')) % 26
        key += chr(shift + ord('a'))
    print(f"\nGuessed key: {key.upper()}")
    return key
def vigenere_decrypt(ciphertext, key_length):
    decrypted_text = ""
    key = vigenere_extract_key(ciphertext, key_length)
    key_index = 0
    for char in ciphertext:
        if char.isalpha():
            shift = ord(key[key_index % key_length]) - ord('a')
            if char.islower():
                decrypted_char = chr(
                    (ord(char) - ord('a') - shift) % 26
                    + ord('a')
                )
            else:
                decrypted_char = chr(
                    (ord(char) - ord('A') - shift) % 26
                    + ord('A')
                )
            decrypted_text += decrypted_char
            key_index += 1
        else:
            decrypted_text += char
    return decrypted_text
ciphertext = """VVHWGQUIVYHIRSUPGTWXJSVIPCWIUPHGCIVIVVHPGHWITSDTRSDVUSYITMZL
GFHIXSUCVSDQOSPFGFVLQIOHEVHGMSDGJQRPWAQWGSNXJSPSUHIVGEXIPHVC
OPRPCBGYUSLXVCUIECYITHKIMSBAJSQXJSPIUGDKGPHGQAHWEZHETSQXGFWL
GOFGGGVAQFGIEZLTUS"""
print(vigenere_decrypt(ciphertext, 4))
```

Output – Groups 1 and 2



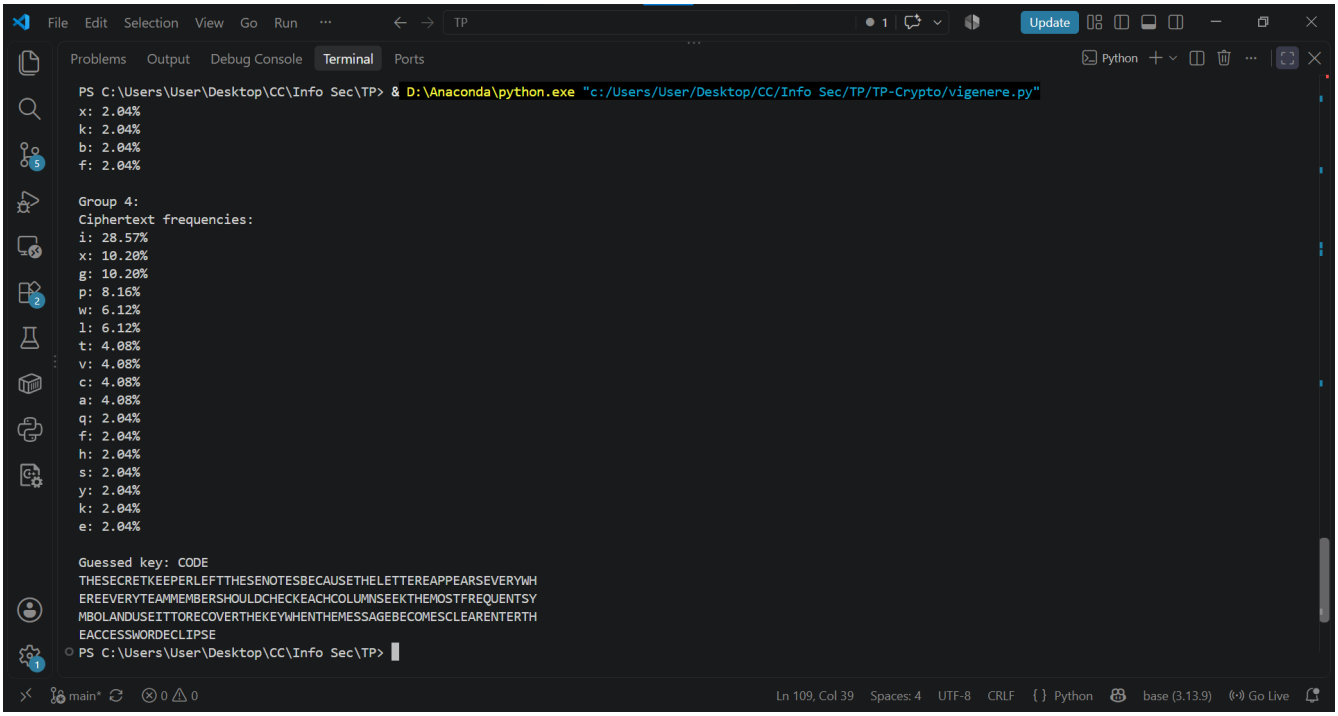
Frequency analysis of groups 1 and 2. **Result.** The most frequent letters are `G` (22%) and `S` (34%), giving shifts 2 and 14, hence key letters `C` and `O`.

Output – Group 3



Frequency analysis of group 3 and the start of group 4. **Result.** Group 3's most frequent letter is `H` (18.37%), giving shift 3 and key letter `D`.

Output – Key and Decrypted Message



Group 4 frequencies, the recovered key, and the decrypted message. **Result.** Group 4's most frequent letter is `I` (28.57%), giving shift 4 and key letter `E`.

Group	Most frequent letter	Assumed plaintext	Shift	Key letter
1	G	E	2	C
2	S	E	14	O
3	H	E	3	D
4	I	E	4	E

Recovered key: `CODE`. **Access word:** `ECLIPSE`.

Conclusion. Caesar is vulnerable to brute force; substitution preserves letter frequencies and word patterns; a Vigenère cipher with a known short repeating key length can be attacked by analyzing each column separately.